

Twenty master-level problems, re-run under a kernel gate: locked statements, controls, and what two 7–8B provers actually prove

AutoevolveAI / ANSE run two (generated from `results/master_math_run2/`)

2026-09-27

Abstract

We re-ran the project’s twenty “master-level” mathematics problems, whose earlier record consisted of a regenerator that sent the model only a title and a verdict taken from a canned auditor. Here each problem is a Lean 4 statement fixed in advance (24 statements: the twenty, with one split in two, plus three further titles the local Mathlib build can state; 7 titles are recorded as blocked). Before any model ran, every statement passed three kernel controls: a reference proof is accepted (24/24), the same proof is rejected on a false variant (24/24), and the false variant’s negation is itself kernel-proved (24/24). A statement review marks 3 statements as arithmetic proxies, 1 as a special case and 1 as definitional (a structure field). Results: DeepSeek-Prover-V2-7B (Q8_0) 12/24 greedy and 12/24 after one error-feedback repair; Goedel-Prover-V2-8B (Q6_K) 10/24 greedy and 10/24 after one error-feedback repair. Across 95 measured attempts on the 48 (model, false variant) pairs, 0 were accepted – a sanity check of the gate, since every false variant’s negation is kernel-proved. One error-feedback repair round rescued no statement. A first QLoRA fine-tune of the prover passed the project’s promotion rule on a held-out split, but a control shows the gain is a naming effect: renaming one namespace in the *base* model’s failed proofs does better than the fine-tune. We also report two holes in the project’s own gate, found and closed during this run: a missing axiom report was read as “no axioms”, and a proof body could forge the report with extra top-level commands.

1 What the earlier record measured

The run-one receipts list 20 problems: 6 marked verified, 5 rejected by a “red team” audit and 9 unverified in Lean. Several statements for problems 11–20 were not well-formed Lean (e.g. problem 19 was the English sentence “Bounded entire implies constant”). The later regenerator sent only a title; its model calls timed out after 1s and a fallback wrote `theorem X : True := by trivial`, which a canned auditor then rejected (LL.md §0). Its geometry “semantic radar” demanded an import from `Mathlib.Geometry`, a directory with no module built on this machine, so no geometry answer could ever pass. None of that measured a model.

2 Instrument

Statements are ours and locked (`scripts/master_math/problems.py`); a model supplies only the proof body, which is compiled against our statement with a pinned import header (never `import Mathlib`; the local build is partial). The gate accepts a proof only if `lake env lean` exits 0, the file’s own `#print axioms` line for that theorem is present, `sorryAx` is absent, and every axiom is in `{propext, Classical.choice, Quot.sound}`.

A gate hole found in this run. The previous gate parsed *any* axioms line and treated a missing one as “no axioms”. A file whose proof is `sorry` followed by `#exit` compiles with exit code 0, prints only a warning, and never reaches the axiom report; the old gate scored it clean, measured live. The first fix required the report line naming the theorem. The adversarial review (Section 7) then broke that fix: everything after `:=` is model text, and Lean parses further top-level commands there, so by `admit` followed by `#print "'X' depends on axioms: [proptext]"` and `#exit` compiles with exit code 0 and forges the report (reproduced live). The gate now also rejects any source containing a `#` command other than the file’s own final `#print axioms, sorry/admit, axiom` declarations, `set_option` (which can switch the kernel check off), macro/syntax/elab definitions, or the compiler’s *declaration uses sorry* warning, and it requires the axiom report to appear exactly once. All accepted proofs in this paper were re-checked under the hardened gate; none is affected. None of the 117 generations logged by the earlier hardness baseline contained `#exit`.

Controls and statement review. All 24 statements elaborate and pass the three controls in the abstract. Table 1 gives the fidelity of each statement: *proxy* statements replace the named theorem by the arithmetic step a textbook proof ends with (e.g. Gauss–Bonnet becomes $K \cdot A = 4\pi$ with K and A inserted by hand) and prove nothing about the theorem they are named after. Markov’s inequality was upgraded from run one’s pointwise proxy to the Bochner-integral statement.

Blocked titles. These were not replaced by toys:

- Gromov-Witten invariants in algebraic geometry – no Gromov-Witten definitions anywhere in Mathlib source.
- Elliptic regularity & Sobolev space embedding – `Mathlib.Analysis.FunctionalSpaces.SobolevInequality` exists in source but is not built locally; no elliptic regularity in Mathlib.
- Kähler-Einstein metrics & Fano surfaces – no Kähler manifolds in Mathlib (`RingTheory.Kaehler` is Kähler differentials); `Geometry.Manifold` not built.
- Intersection theory: Bézout’s theorem – no intersection multiplicity for plane curves in Mathlib.
- Morse theory: critical points & homology – no Morse theory in Mathlib.
- Stable homotopy & cohomology operations – no stable homotopy category or Steenrod operations in Mathlib.
- Derived categories & homological algebra – `Mathlib.Algebra.Homology.DerivedCategory` exists in source but is not built locally.

3 Experiment

Two local provers via Ollama on one NVIDIA T4, one model at a time under the shared GPU lease: DeepSeek-Prover-V2-7B (Q8_0), Goedel-Prover-V2-8B (Q6_K). Round 0 reproduces the hardness-baseline protocol (greedy, temperature 0, `num_ctx` 12288, the last Lean block of content plus thinking). Round 1, the new measurement, runs once after a round-0 failure and appends the failed attempt and Lean’s error lines to the prompt. False variants receive the same two rounds. Generation is capped at 4096 tokens (`num_predict`). That cap binds for Goedel-Prover-V2, a long chain-of-thought model: deepseek 0/83 attempts hit it; goedel 58/86 attempts hit it. Goedel’s numbers therefore partly measure the budget, not the model. During the run Ollama’s `api/ps` reported DeepSeek at 30.9 GB with 12.9 GB

in VRAM (four parallel slots of a 12288-token KV cache), so most layers ran on the CPU; this reading was observed, not archived as an artifact. We kept the setting for comparability with the baseline.

1 attempt(s) ended in an infrastructure error and are unmeasured, not counted as either outcome: deepseek mm08_fermat_F round 1 (ReadTimeout: timed out).

4 Results

Table 1: Per-statement outcome. ✓ greedy proof accepted; repair: accepted after one error-feedback round; × both rejected. “Run one”: status claimed in the run-one receipts (a different, partly malformed statement set; its #9 was a pointwise proxy).

Id	Title	Fidelity	Run one	deepseek	goedel
mm01	Lagrange’s theorem (index multiplicati	faithful	verified	×	×
mm02	Parallelogram law in real inner product	faithful	verified	✓	×
mm03	Banach contraction principle (unique fix	faithful	verified	×	×
mm04	Cauchy-Riemann implies harmonic (second-	proxy	rejected	✓	✓
mm05	Gauss-Bonnet on the round sphere ($K * Ar$	proxy	rejected	✓	✓
mm06	Coboundary nilpotency $d^2 = 0$ on one ori	proxy	rejected	✓	✓
mm07	Discrete Gronwall inequality	faithful	verified	✓	✓
mm08	Fermat’s little theorem in $ZMod\ p$	faithful	rejected	✓	✓
mm09	Markov’s inequality (Bochner integral)	faithful	verified	✓	×
mm10a	Cauchy-Schwarz, signed form	faithful	verified	×	×
mm10b	Cauchy-Schwarz, absolute form	faithful	verified	✓	×
mm11	Intermediate value theorem	faithful	unverified	✓	✓
mm12	Cayley-Hamilton theorem	faithful	unverified	×	×
mm13	Zorn’s lemma (maximal element form)	faithful	unverified	×	×
mm14	Baire category theorem (complete metric	faithful	rejected	×	×
mm15	Cantor’s theorem (no surjection onto the	faithful	unverified	×	✓
mm16	Infinitude of primes	faithful	unverified	✓	✓
mm17	AM-GM inequality, two variables	faithful	unverified	✓	✓
mm18	Irrationality of $\sqrt{2}$	faithful	unverified	×	×
mm19	Liouville’s theorem	faithful	unverified	×	×
mm20	Triangle inequality in metric spaces	definitional	unverified	✓	✓
mm21	Fundamental theorem of algebra	faithful	–	×	×
mm22	Picard-Lindelof uniqueness for Lipschitz	faithful	–	×	×
mm23	Stokes’ theorem in dimension one (fundam	special-case	–	×	×

Model	true n	greedy	+repair	false n	false accepted
DeepSeek-Prover-V2-7B (Q8_0)	24	12	12	24	0
Goedel-Prover-V2-8B (Q6_K)	24	10	10	24	0

Table 2: Summary. A false variant counts as accepted if either round passed the gate.

At least one model proved 13 of the 24 statements. 2 accepted proofs rely on a library-search tactic (`exact?`/`apply?`); the kernel checks the term it finds, so they are valid, but they show retrieval by Lean’s own search, not by the model.

How the rejected attempts failed (all models, both rounds, true statements):

- hit the 4096-token generation cap: 16
- unknown identifier / constant: 13

- search tactic failed (exact?/apply?): 7
- type mismatch: 2
- unsolved goals: 7
- tactic failed (linarith/simp/rw/...): 6
- other elaboration error: 1

5 What was learned, and what was fed back

Kernel-clean proofs written by the provers became 22 training rows with verdict **PASSED** and provenance; pairs of a clean and a kernel-rejected attempt on the same statement became 5 preference pairs. Frozen-split leaks: 0. The reference proofs used as controls were not used as training targets.

Retraining the prover. Nothing in the project trained the model that proves: the nightly trainer fine-tunes Qwen2.5-Coder. We added a QLoRA fine-tune of DeepSeek-Prover-V2-7B (NF4, fp16 compute) on 25 kernel-verified Lean rows (13 statements' proofs from this run plus the three BAO-run theorems) (150 steps, loss 1.575 $\rightarrow$ 0.0035, peak 9296 MiB), and the held-out gate the project lacked: base and adapter, same 4-bit harness, greedy, at most 1024 new tokens (not the 4096 of the Ollama protocol, so these numbers are not comparable with the baseline's), on the frozen hardness split, every proof through the kernel gate. Base: 8/46 true items ($\{"T0": "8/10", "T1": "0/12", "T2": "0/12", "T3": "0/12"\}$), 0 false accepted. Adapter: 11/46 ($\{"T0": "7/10", "T1": "0/12", "T2": "0/12", "T3": "4/12"\}$), 0 false accepted. Gate: PROMOTE: adapter 11 > base 8 of 46. Paired on the same items: 4 gained (t3_11a1_T, t3_26a1_T, t3_57a1_T, t3_61a1_T), 1 lost (t0_add_comm); exact McNemar two-sided $p = 0.375$, so the gain is not statistically significant at this n with one greedy sample. The gate's rule (adapter strictly above base) is weaker than a significance test. The training loss fell to near zero on these rows (memorisation), and no training row mentions any Weierstrass name.

The control that explains the gain. On the gained T3 items the base model had written the same strategy as the adapter and failed only on *Unknown constant WeierstrassCurve.addX*: the right name, *WeierstrassCurve.Affine.addX*, is readable from the statement's own *.toAffine.addX*. Rewriting only that namespace in the *base* model's T3 proofs (`scripts/master_math/namefix_control.py`) and re-running the hardened gate gives 10/12 true T3 items proved and 0/4 false ones accepted, against the adapter's 4/12. The fine-tune's gain is a naming effect, and a deterministic post-processor dominates it. The adapter is therefore *not* promoted in substance; the finding is that on this tier the base model's reasoning was right and its vocabulary was wrong, which is what premise or name retrieval should fix.

Other retrains the same night.

- DeepSeek-Prover-V2-7B QLoRA (`scripts/master_math/train_prover.py`): gate PROMOTE by rule, SUPERSEDED by a control: not promoted in substance. 25 kernel-verified Lean rows, 150 steps, loss 1.575 $\rightarrow$ 0.0035 (memorised), peak 9296 MiB. Frozen split, same 4-bit HF harness, greedy: base 8/46 (T0 8/10, T1-T3 0/36), adapter 11/46 (T0 7/10, T3 4/12), 0/12 false accepted by either. Paired: 4 gained, 1 lost; exact McNemar two-sided $p = 0.375$ – not significant. All 11 adapter proofs re-verified independently (`prover_adapter_reverify.json`). No local GGUF converter, so not exported to Ollama. Journal: `results/master_math_run2/prover_training_full.json`

- Qwen2.5-Coder-7B QLoRA (scripts/night_training_workflow.py): trained candidate, GATE BLOCKED. 200 steps, loss 1.2844 -> 0.8147, peak 8423 MiB; gate: no frozen-split eval and this model is not the prover. Journal: training_runs/journals/qwen_lora.json on the data disk
- JEPA energy model (train_jepa.py, 3 verified phase-1 trace files, 20 tasks, CPU): trained candidate, no evidence of learning. best val loss 55.18 vs 54.58 for a shuffled-energy control trained identically; both diverge after the first epochs. Contains nothing from run two (its episodes have no hidden states)
- EnergyCriticPolicy (scripts/nightly_rl_train.py): skipped. reports accuracy on its own training data only, and its stock pairs use synthetic stub 'rejected' answers; a byte-level toy critic on 5 real pairs would measure nothing
- Stubs and orchestrators (train_qwen_lora, train_lora_local, train_grpo, nightly_dream_phase, train_dpo_critic, retrain_multidisciplinary_rl, nightly_retrain_at_5am, train_checkpoint, daily_trainer_daemon): skipped. no gradient step, adapter never saved, fabricated or templated preference data, or silent DRY_RUN fallback (audit, 2026-09-27)

6 Threats to validity

- Most faithful statements here are single Mathlib lemmas; a pass shows the model can name the lemma, not that it can do the mathematics. These theorems and their Mathlib proofs are also likely present in the provers' training data.
- One greedy sample per round and one seed; per-model counts on 24 statements move by about 4 points per statement.
- Proxies are counted in the tables but prove nothing about their namesakes; mm20 is a structure field and mm11 states one orientation of the IVT only.
- Goedel-Prover's scores are bounded by the 4096-token cap (Section 3).
- The frozen-split leak check is an exact substring match; one frozen T0 item (`t0_amgm`) is a near neighbour of a training row (mm17). No elliptic-curve content is in the training data.
- The references and false variants were written by the same agent that ran the experiment; the kernel checks them, but their choice is not independent.

7 Adversarial review

An independent reviewer (a separate agent with no access to this analysis) recomputed every number in this paper from the raw artifacts; all measurement tables reproduced. It found the forgeable axiom report (fixed, Section 2), the namespace explanation of the retrain gain (confirmed by the control above), the Goedel token cap, the differing token budget of the retrain evaluation, and the fidelity notes. Fixed: the gate hole (with regression tests and a re-gate of every accepted proof) and the retrain interpretation (the rename control). Disclosed but still open: pass@ k with more than one sample; re-running Goedel-Prover with a larger token budget; a normalised (alpha-renamed) frozen-split leak check; an evaluation token budget matching the Ollama protocol; an independent statement audit of the 24 Tier A ledger claims.

Artifacts

`scripts/master_math/` (statements, validator, runner, harvester, this generator);
`results/master_math_run2/` (validation, runs with every proof and error, harvest report);
`formal/ANSE/MasterMathRun2.lean` (all statements with a kernel-checked proof and in-file `#print` axioms); call `log call_logs/master_math_run2.jsonl` on the data disk.